Name & Surname **Polimi ID**

Exercise 1

A. How does the Fork-Join pattern differ from a barrier in parallel programming? (1)

B. How does the Workpile pattern differ from a standard Map pattern? (1)

C. Consider the following nested loop:

```
for (j = 0; j < 100; j++) {
    for (i = 1; i < 100; i++) {
        a[i][j] = f(a[i-1][j]);
    }
}
```

Evaluate the cache-friendliness of this code. How does the memory layout of the array affect performance, and what changes could improve cache utilization? Is it still parallelizable? How? Justify your answer. (2)

Name & Surname **Polimi ID**

Exercise 2

A. How can the map and reduce patterns be fused, and what is the benefit of doing so? (1)

B. In the context of parallel scan, what is the difference between an inclusive and an exclusive scan? What about the complexity? (1)

C. Please discuss how the Split parallel pattern works. How is it implemented (pseudo code required and PRAM-based complexity)? (2)

Name & Surname **Polimi ID**

Exercise 3

A. It is always necessary to place an explicit barrier at the end of an OpenMP work sharing construct. Is this statement true or false? Explain. (1)

B. List the factors that determine the number of threads in an OpenMP parallel region. (1) **This question can be skipped if you got 1 or 3 points in the second challenge.**

C. Describe the effects of the following pragma. (2) **This question can be skipped if you got 2 or 3 points in the first challenge.**

```
#pragma omp task depend(in: status[i])
```

Name & Surname **Polimi ID**

Exercise 4

A. Modern HPC systems feature cores in the order of millions. Why can MPi help my application to use more cores than a plain OpenMP application?(1)

B. Make an **example** where the program's correct execution **requires** `MPI_probe` (1)

C. MPI functions with non-blocking semantics, e.g. MPI_Irecv, and multi-threading programming, e.g. OpenMP, can be used to solve the same problem. What is the problem and how can the two approaches be used to solve it? (2)